

Drawlib Quickstart Guide

Illustration as Code & Documentation as Code in Pure Python

Drawlib is a modern Python library for creating technical diagrams, architecture blueprints, charts, SmartArts, and publication-ready documentation directly from code.

```

from drawlib.canvas import config
from drawlib.colors import Colors, Colors140
from drawlib.icons import gcp, phosphor
from drawlib.lines import line
from drawlib.shapes import circle, rectangle
from drawlib.text import text
from drawlib.types import Style

config(width=120, height=50)

# Left card: Python Code
rectangle(
    xy=(22, 25),
    width=30,
    height=32,
    r=3,
    style=Style(fill_color=Colors140.AliceBlue, line_color=Colors140.RoyalBlue, line_width=2),
)
phosphor.code(xy=(22, 31), width=10, style="blue")
text(xy=(22, 16), text="Python Code", style=Style(text_color=Colors.Navy, text_size=12))

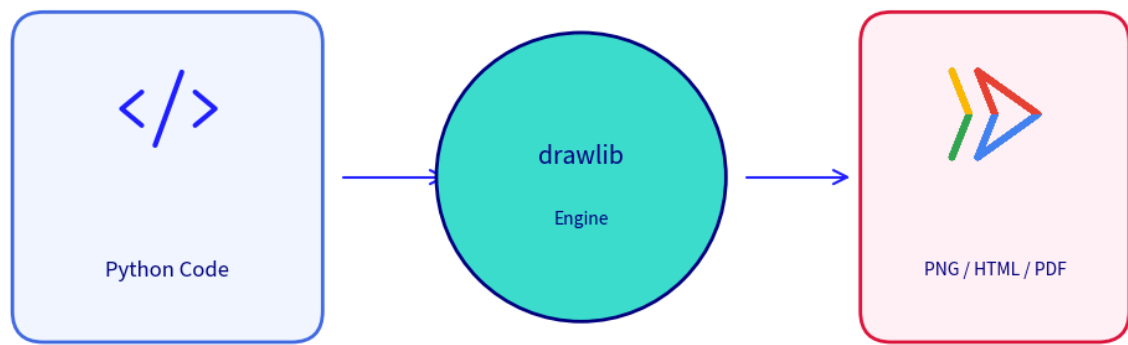
# Arrow 1
line((39, 25), (49, 25), arrowhead="->", style="blue")

# Center card: Drawlib Engine
circle(
    xy=(62, 25),
    radius=14,
    style=Style(fill_color=Colors140.Turquoise, line_color=Colors.Navy, line_width=2),
)
text(xy=(62, 27), text="drawlib", style=Style(text_color=Colors.Navy, text_size=14))
text(xy=(62, 21), text="Engine", style=Style(text_color=Colors.Navy, text_size=10))

# Arrow 2
line((78, 25), (88, 25), arrowhead="->", style="blue")

# Right card: Output Formats
rectangle(
    xy=(102, 25),
    width=26,
    height=32,
    r=3,
    style=Style(fill_color=Colors140.LavenderBlush, line_color=Colors140.Crimson, line_width=2),
)
gcp.cloud_run(xy=(102, 31), width=10)
text(xy=(102, 16), text="PNG / HTML / PDF", style=Style(text_color=Colors.Navy, text_size=10))

```



Drawlib: Illustration as Code in Pure Python

About This Document

This quickstart guide introduces the core workflows of **Drawlib**, from basic canvas coordinates and preset styling to icons, SmartArts, declarative charts, software diagrams, and the unified CLI document builder.

- **Repository:** <https://github.com/yuichil10/drawlib>
- **Author:** Yuichi Ito
- **License:** Apache License, Version 2.0

Table of Contents

1. About Drawlib

2. Installation & Setup

3. Canvas, Coordinates & Shapes

4. Preset Styles & Themes

5. Icons: Phosphor & Google Cloud (GCP)

6. Unified CLI & Document Builder

7. SmartArts Components

8. Declarative Charts

9. Software & Architecture Diagrams

1. About Drawlib

Drawlib is a pure Python drawing library designed to facilitate **Illustration as Code**. Instead of manually positioning shapes in GUI drawing tools that store opaque binary files, Drawlib generates crisp illustrations directly from readable, version-controlled Python scripts and Markdown documents.

Why Illustration as Code?

1. **Git & Code Review Friendly:** Every diagram is plain Python text. Track changes with `git diff`, review architecture updates in pull requests, and automate builds in CI/CD.
2. **Visual Consistency via Themes:** Apply unified preset styles (`default` , `essentials` , `monochrome` , or custom themes) across hundreds of illustrations with zero manual formatting.
3. **Full Power of Python:** Use loops, helper functions, data structures, and full IDE type hints / autocompletion instead of learning a restricted Domain-Specific Language (DSL).

First Example

Here is a simple example combining a styled circle, rectangle, and text label on a Drawlib canvas:

```
from drawlib.canvas import config, save
from drawlib.colors import Colors140
from drawlib.shapes import circle
from drawlib.types import Style

config(width=100, height=50)
circle(
    xy=(50, 25),
    radius=18,
    style=Style(
        line_style="dashed",
        line_color=Colors140.BlueViolet,
        line_width=3,
        fill_color=Colors140.Turquoise,
    ),
)
save()
```

When compiled by Drawlib, this code renders the following illustration:

```
from drawlib.canvas import config
from drawlib.colors import Colors140
from drawlib.shapes import circle
from drawlib.text import text
from drawlib.types import Style

config(width=100, height=50)
circle(
    xy=(50, 25),
    radius=18,
    style=Style(
        line_style="dashed",
        line_color=Colors140.BlueViolet,
        line_width=3,
        fill_color=Colors140.Turquoise,
    ),
)
text(xy=(50, 25), text="Hello drawlib!", size=14)
```

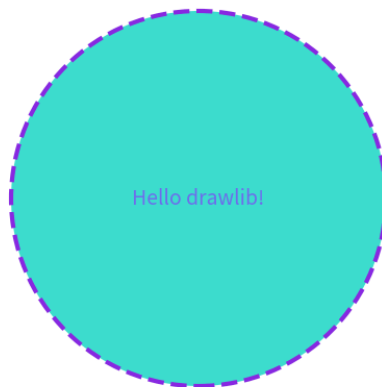


Figure 1.1: Styled Circle Generated from Python Code

2. Installation & Setup

Drawlib requires **Python 3.11 or higher** and can be installed from PyPI using `pip` or `uv`.

Installing with pip or uv

```
# Using pip
pip install drawlib

# Using uv
uv add drawlib
```

Verifying the Installation

Once installed, both the Python package `drawlib` and the `drawlib` command-line tool are available:

```
$ drawlib --version
```

You can also invoke the CLI via Python's module runner:

```
$ python -m drawlib --version
```

Semantic Versioning & Stability

Drawlib follows `<major>.<minor>.<patch>` semantic versioning:

- **Major Version:** Significant architectural or API changes
- **Minor Version:** New features and backward-compatible enhancements
- **Patch Version:** Bug fixes and internal optimizations without API changes

```

from drawlib.canvas import config
from drawlib.colors import Colors, Colors140
from drawlib.shapes import chevron
from drawlib.text import text
from drawlib.types import Style

config(width=110, height=36)

steps = [
    ("1. Install", "pip / uv", Colors140.CornflowerBlue),
    ("2. Author", "Python / MD", Colors140.MediumSeaGreen),
    ("3. Preview", "drawlib show", Colors140.GoldenRod),
    ("4. Publish", "HTML / PDF", Colors140.Tomato),
]

for idx, (title_str, sub_str, fill_col) in enumerate(steps):
    cx = 16 + idx * 26
    chevron(
        xy=(cx, 18),
        width=23,
        height=18,
        corner_angle=50,
        style=Style(fill_color=fill_col, line_color=Colors.White, line_width=1.5),
    )
    text(xy=(cx, 20), text=title_str, style=Style(text_color=Colors.White, text_size=11))
    text(xy=(cx, 14), text=sub_str, style=Style(text_color=Colors.White, text_size=9))

```



Figure 2.1: Drawlib Release & Versioning Workflow

3. Canvas, Coordinates & Shapes

Every Drawlib illustration takes place on a coordinate **Canvas**. By default, the canvas spans `width=100` and `height=100` coordinate units with the origin `(0, 0)` at the bottom-left corner.

Configuring the Canvas

Use `config()` from `drawlib.canvas` to customize the coordinate dimensions, resolution (`dpi`), or alignment grid (`grid=True`):

```
from drawlib.canvas import config

config(width=100, height=50, dpi=200, grid=True)
```

- `width` / `height` : Logical coordinate space (default: `100` x `100`).
- `dpi` : Dots per inch resolution (default: `100` , where 10 inches x 100 DPI = 1000px width).
- `grid` : Overlays a coordinate grid and center axes to assist with precise placement.

Drawing Fundamental Primitives

Drawlib provides intuitive functions for lines, circles, rectangles, polygons, arrows, and text:

```
from drawlib.canvas import config
from drawlib.lines import line
from drawlib.shapes import arrow, circle, rectangle
from drawlib.text import text

config(width=100, height=50, grid=True)

# Circle at (20, 28)
circle(xy=(20, 28), radius=12, style="blue")
text(xy=(20, 8), text="circle((20, 28))", size=10)

# Rectangle at (50, 28)
rectangle(xy=(50, 28), width=22, height=20, r=2, style="green")
text(xy=(50, 8), text="rectangle((50, 28))", size=10)

# Arrow and Line at (80, 28)
arrow((70, 28), (90, 28), tail_width=4, head_width=10, head_length=6, style="red")
line((70, 18), (90, 18), arrowhead="<->", style="black")
text(xy=(80, 8), text="arrow & line", size=10)
```

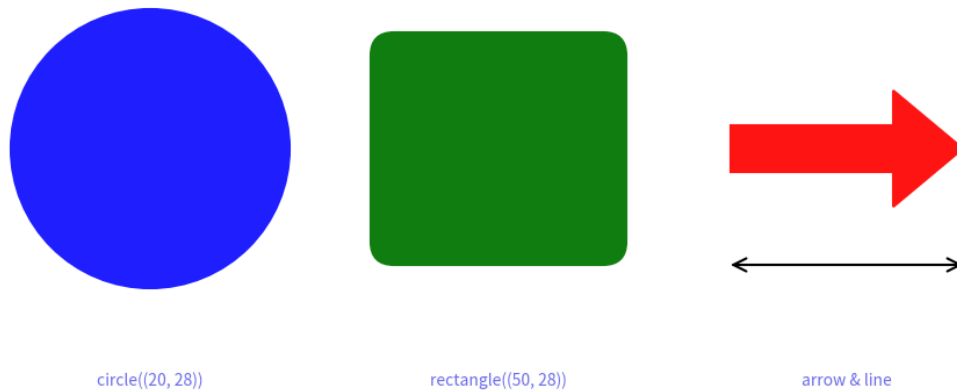


Figure 3.1: Fundamental Shapes and Coordinate Grid

Coordinate Alignment (`halign` and `valign`)

All shapes and text elements support horizontal (`left` , `center` , `right`) and vertical (`bottom` , `center` , `top`) alignment via `Style(halign=..., valign=...)` , making it effortless to align labels and boxes around any anchor point `(x, y)` .

4. Preset Styles & Themes

Instead of constructing a custom `Style(...)` object for every shape, Drawlib includes a powerful **Preset Styles** system (`drawlib.preset_styles`). Every drawing function accepts a shorthand style name string such as `style="blue"` or `style="red_flat"` .

Official Style Presets

Drawlib ships with three curated preset themes:

- `"default"` : Clean, approachable palette with soft fills and crisp borders.
- `"essentials"` : Rich, high-contrast palette for modern technical presentations.
- `"monochrome"` : Grayscale palette tailored for print books and academic papers.

Using Style Name Shortcuts

You can combine color names (`blue` , `green` , `red` , `black` , `white`) with variant modifiers (`flat` , `solid` , `dashed`) directly in the `style` parameter:

```
circle((20, 25), radius=10)           # Default style
circle((45, 25), radius=10, style="blue") # Blue accent
circle((70, 25), radius=10, style="green") # Green accent
```

```

from drawlib.canvas import config
from drawlib.lines import line
from drawlib.shapes import circle
from drawlib.text import text

config(width=100, height=45)
line_y = 36
text_y = 9

# 1. Default style
line((12, line_y), (26, line_y))
circle((19, 23), radius=8)
text((19, text_y), text="default", size=10)

# 2. Blue style
line((33, line_y), (47, line_y), style="blue")
circle((40, 23), radius=8, style="blue")
text((40, text_y), text='style="blue"', style="blue", size=10)

# 3. Green style
line((54, line_y), (68, line_y), style="green")
circle((61, 23), radius=8, style="green")
text((61, text_y), text='style="green"', style="green", size=10)

# 4. Red style
line((75, line_y), (89, line_y), style="red")
circle((82, 23), radius=8, style="red")
text((82, text_y), text='style="red"', style="red", size=10)

```

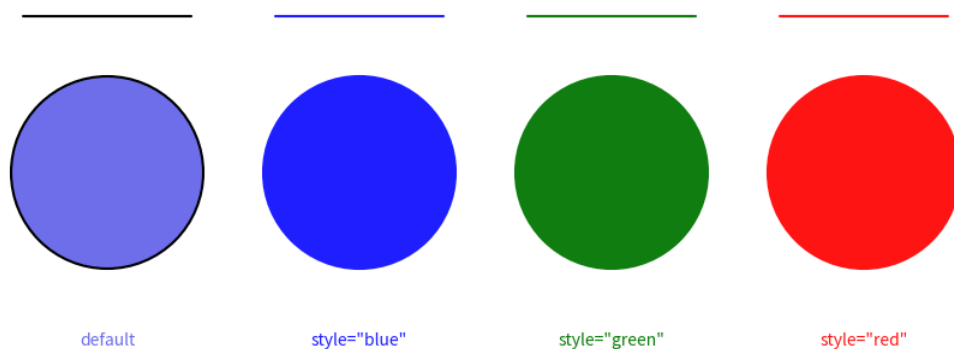


Figure 4.1: Applying Preset Style Names to Lines, Shapes, and Text

5. Icons: Phosphor & Google Cloud (GCP)

Drawlib bundles rich icon modules under `drawlib.icons` so you can build expressive system architecture diagrams without hunting for external image assets.

1. Phosphor Icons (`drawlib.icons.phosphor`)

`phosphor` provides ~1,500 scalable vector font icons covering devices, networking, users, arrows, and UI metaphors. Each icon is a Python function accepting `xy` , `width` , `angle` , and `style` :

```
from drawlib.canvas import config
from drawlib.icons import phosphor
from drawlib.text import text

config(width=100, height=42)

items = [
    (16, phosphor.desktop, "desktop", "blue"),
    (38, phosphor.database, "database", "green"),
    (60, phosphor.cloud, "cloud", "blue"),
    (84, phosphor.shield_check, "shield_check", "red"),
]

for x, fn, label, st in items:
    fn(xy=(x, 26), width=11, style=st)
    text(xy=(x, 10), text=f"phosphor.{label}", size=9)
```



`phosphor.desktop`



`phosphor.database`



`phosphor.cloud`



`phosphor.shield_check`

Figure 5.1: Vector Icons from `drawlib.icons.phosphor`

2. Google Cloud Platform Icons (drawlib.icons.gcp)

drawlib.icons.gcp provides 250+ official multi-color Google Cloud service and category icons designed for cloud architecture blueprints:

```
from drawlib.canvas import config
from drawlib.icons import gcp
from drawlib.lines import line
from drawlib.text import text

config(width=100, height=42)

services = [
    (16, gcp.cloud_load_balancing, "Load Balancing"),
    (40, gcp.cloud_run, "Cloud Run"),
    (64, gcp.cloud_sql, "Cloud SQL"),
    (86, gcp.bigquery, "BigQuery"),
]

for idx, (x, fn, label) in enumerate(services):
    fn(xy=(x, 26), width=11)
    text(xy=(x, 10), text=label, size=9)
    if idx < len(services) - 1:
        next_x = services[idx + 1][0]
        line((x + 8, 26), (next_x - 8, 26), arrowhead="->", style="blue")
```

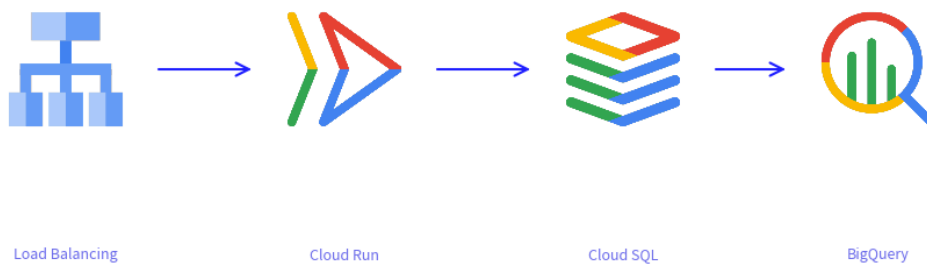


Figure 5.2: Official Google Cloud Service Icons from drawlib.icons.gcp

6. Unified CLI & Document Builder

Drawlib includes a built-in **Document Builder** and **CLI** (`drawlib`) that compiles standalone Python scripts or Markdown documentation containing embedded `drawlib` code blocks into multiple publication targets.

Core CLI Subcommands

Command	Description
<code>drawlib build image <src></code>	Execute <code>.py</code> files or directories to batch-generate illustration images.
<code>drawlib build markdown <src> -o <out></code>	Compile Markdown with <code>drawlib</code> blocks into standard GitHub-ready Markdown + images.
<code>drawlib build html <src> -o <out></code>	Compile Markdown/HTML into a responsive static HTML website (<code>--css google</code>).
<code>drawlib build pdf <src> -o <out.pdf></code>	Merge Markdown/HTML files in filename order into a single PDF (<code>--generate-index</code>).
<code>drawlib show <file> [block]</code>	Preview an illustration or Markdown block interactively or headlessly.
<code>drawlib export <file> [block] -o <img.png></code>	Export a specific <code>drawlib</code> block from Markdown to an image file.
<code>drawlib serve <html_dir></code>	Start a local HTTP server with link and asset verification.

Example Build Commands

```
# Build GitHub Markdown and Google-styled HTML website
uv run drawlib build markdown docs_src/ -o docs/
uv run drawlib build html docs_src/ -o docs_html/ --css google

# Build merged Quickstart PDF with auto-generated index between 1st and 2nd files
uv run drawlib build pdf docs_src_quickstart_pdf/ -o quickstart.pdf --generate-index --css google
```

```

from drawlib.canvas import config
from drawlib.colors import Colors, Colors140
from drawlib.lines import line
from drawlib.shapes import rectangle
from drawlib.text import text
from drawlib.types import Style

config(width=110, height=46)

# Source
rectangle(xy=(20, 23), width=28, height=24, r=2, style=Style(fill_color=Colors140.AliceBlue,
line_color=Colors140.RoyalBlue, line_width=2))
text(xy=(20, 26), text="Markdown + drawlib", size=10)
text(xy=(20, 19), text="(docs_src/)", size=9)

# Compiler
rectangle(xy=(56, 23), width=24, height=20, r=2, style=Style(fill_color=Colors140.Turquoise,
line_color=Colors.Navy, line_width=2))
text(xy=(56, 23), text="drawlib build", size=10)

line((34, 23), (44, 23), arrowhead="->", style="blue")

# Outputs
outputs = [
    (36, "docs/ (Markdown)", Colors140.HoneyDew),
    (23, "docs_html/ (Web)", Colors140.LemonChiffon),
    (10, "quickstart.pdf (PDF)", Colors140.LavenderBlush),
]
for y_pos, label, col in outputs:
    line((68, 23), (78, y_pos), arrowhead="->", style="blue")
    rectangle(xy=(93, y_pos), width=28, height=9, r=1.5, style=Style(fill_color=col,
line_color=Colors.Navy, line_width=1.5))
    text(xy=(93, y_pos), text=label, size=9)

```

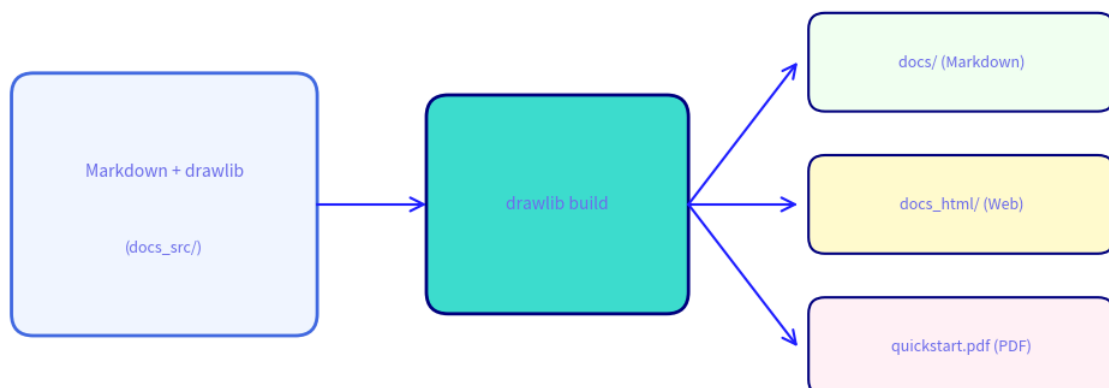


Figure 6.1: Single-Source Documentation Compilation Pipeline

7. SmartArts Components

`drawlib.smartarts` provides high-level presentation graphics inspired by PowerPoint SmartArt. Instead of calculating geometry manually, you instantiate a SmartArt component, add items, and call `.draw()`.

Available SmartArt Classes

- `Pyramid` : Hierarchical pyramid layers with customizable orientation and styles.
- `BoxList` / `ChevronProcess` : Sequential process blocks and arrowhead pipelines.
- `Cycle` / `MindMap` / `Tree` : Circular workflows, mind maps, and hierarchical trees.
- `Table` / `GridLayout` / `SourceCode` : Structured tables, grid cards, and syntax-highlighted code blocks.

Example: Hierarchical Pyramid

```
from drawlib.canvas import config
from drawlib.smartarts import Pyramid

config(width=100, height=48)

p1 = Pyramid()
p1.add(text="Strategy")
p1.add(text="Architecture")
p1.add(text="Implementation")
p1.draw((6, 5), width=40, height=38, margin=2.5)

p2 = Pyramid(default_style="solid")
p2.add(text="Vision", style="blue_flat", textstyle="white")
p2.add(text="Platform", style="green_flat", textstyle="white")
p2.add(text="Operations", style="red_flat", textstyle="white")
p2.draw((54, 5), width=40, height=38, margin=2.5, align="left")
```

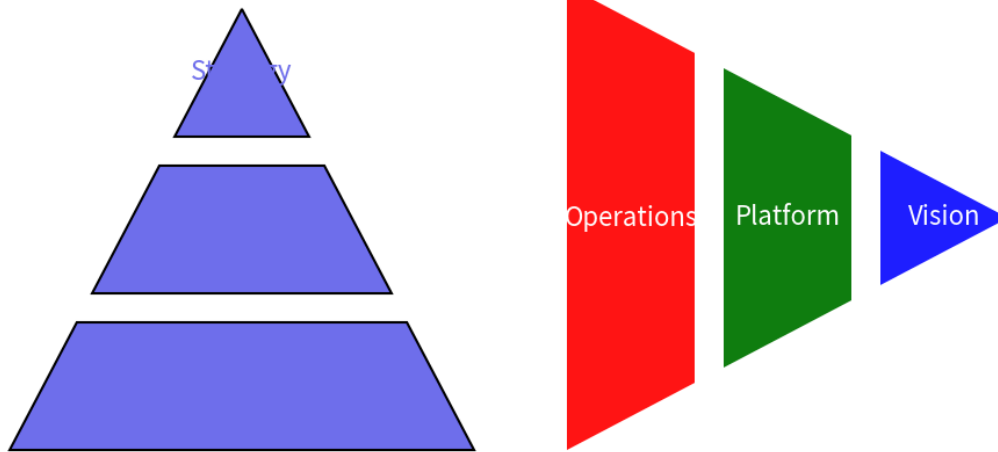


Figure 7.1: SmartArt Pyramid Comparing Default and Custom Styled Layers

8. Declarative Charts

`drawlib.charts` provides vector-grade data visualization built directly on Drawlib's canvas primitives. You can place charts alongside annotations, icons, or architecture diagrams on any canvas.

Supported Chart Types

- `BarChart` : Vertical and horizontal grouped/stacked bars with linear or logarithmic scales.
- `LineChart` & `AreaChart` : Trend lines, smooth splines, custom markers, and stacked areas.
- `PieChart` & `RadarChart` : Proportional slices, donut rings, and multivariate spider charts.
- `ScatterChart` & `GanttChart` : 2D correlation plots and project schedule timelines.

Example: Grouped Bar Chart

```
from drawlib import canvas
from drawlib.charts import BarChart

canvas.initialize()

chart = BarChart(
    categories=["Q1", "Q2", "Q3", "Q4"],
    width=80,
    height=56,
    title="Quarterly Cloud vs On-Premises Throughput",
    show_values=True,
)

chart.add_series("Cloud Native", [48.0, 72.0, 96.0, 128.0])
chart.add_series("Legacy On-Prem", [52.0, 46.0, 39.0, 32.0])
chart.configure_y_axis(unit="req/s", show_grid=True)
chart.draw(xy=(10.0, 18.0))
```

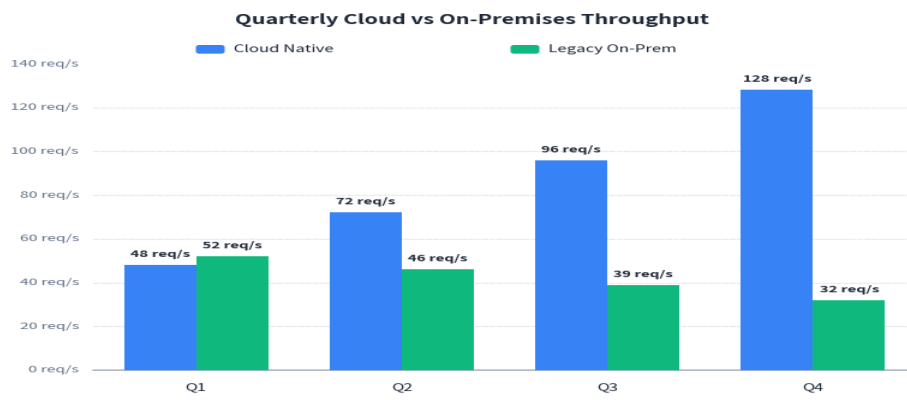


Figure 8.1: Declarative Grouped BarChart

9. Software & Architecture Diagrams

`drawlib.diagrams` provides specialized declarative engines for software engineering and cloud architecture documentation, including **Sequence Diagrams**, **Architecture Diagrams**, **Flow Diagrams**, **Class Diagrams**, **State Diagrams**, and **ER Diagrams**.

Sequence Diagrams (`drawlib.diagrams.sequence`)

With `SequenceDiagram`, participants and lifelines are defined as Python objects and interactions use expressive methods (`request()`, `reply()`) with automatic timeline spacing:

```
from drawlib import canvas
from drawlib.diagrams.sequence import Participant, PhosphorIcon, SequenceDiagram

canvas.initialize()

d = SequenceDiagram(title="API Gateway & Microservice Authentication")

user = d.add(Participant("Client", icon=PhosphorIcon.USER, icon_size=8.0))
gateway = d.add(Participant("API Gateway", icon=PhosphorIcon.CLOUD, icon_size=8.0))
service = d.add(Participant("Auth Service", icon=PhosphorIcon.CPU, icon_size=8.0))
db = d.add(Participant("User DB", icon=PhosphorIcon.DATABASE, icon_size=8.0))

user.request(gateway, "POST /v1/token")
gateway.request(service, "Verify Credentials")
service.request(db, "SELECT user_hash")
db.reply(service, "Record Matched")
service.reply(gateway, "Signed JWT")
gateway.reply(user, "200 OK (JWT)")

d.draw(xy=(5.0, 5.0))
```

API Gateway & Microservice Authentication

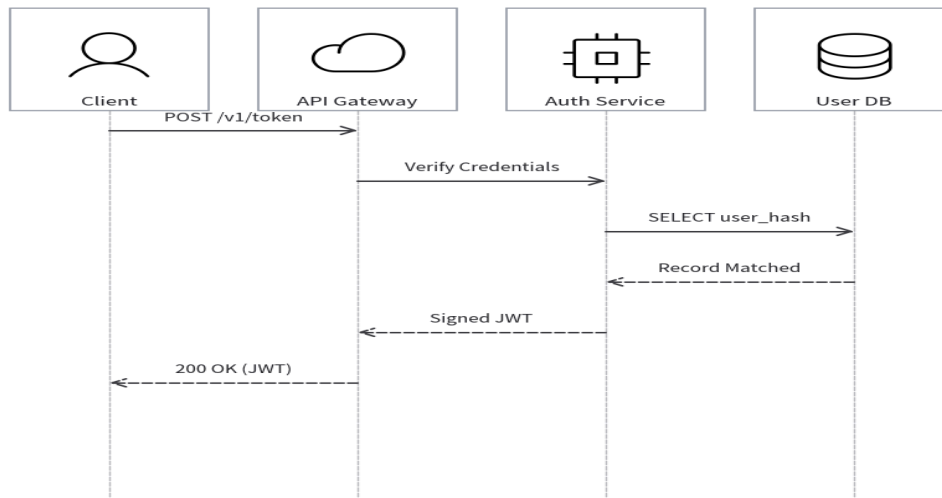


Figure 9.1: Declarative Sequence Diagram with Phosphor Icons